

STM32 Microcontroller

Why an ARM Processor?

- **Engineering:** ARM processors provide efficient and flexible architecture for modern embedded system design.
- **IoT research:** ARM enables low-power, high-performance computing ideal for IoT devices.
- **Economy of Bangladesh:** ARM skills support local tech growth, outsourcing, and semiconductor workforce development.
- **Industry machinery:** ARM controls automation systems, robotics, and smart industrial equipment efficiently.
- **Program almost all machines around us:** Most smart devices (phones, appliances, sensors) run on ARM-based processors.
- **High-level hardware (Trillion-dollar market):** ARM architecture powers a massive global semiconductor ecosystem worth trillions of dollars.

ARM Cortex-M4

1. 32-bit Microprocessor with FPU Core

A microcontroller that processes **32-bit data** and includes a **Floating Point Unit (FPU)** for fast mathematical calculations.

2. Flash Memory & RAM (On-board)

Built-in **Flash memory** stores programs permanently, while **RAM** stores temporary data during program execution.

3. Peripheral Ports / Devices

Interfaces such as **GPIO, UART, SPI, I2C, ADC**, etc., are used to connect sensors, motors, displays, and other external hardware.

Data Size Terminology

- **Byte:** 8-bit data
- **Half-word:** 16-bit data
- **Word:** 32-bit data (native size for Cortex-M4)
- **Double word:** 64-bit data

Key Features of the ARM Cortex-M4 Core in STM32F446RE

1. Core Architecture***

- **ARM Cortex-M4 (RISC Processor):**
A **32-bit RISC (Reduced Instruction Set Computing) processor**, meaning it uses simple and efficient instructions for faster execution and lower power consumption.
- **DSP Instructions:**
The Cortex-M4 includes **Digital Signal Processing (DSP) instructions** that improve performance in applications requiring complex mathematical computations.
- **Floating Point Unit (FPU):**
Supports **single-precision floating-point operations**, meaning it can efficiently handle decimal (real-number) calculations using **32-bit floating-point values**.

2. Performance***

- **Clock Speed:**
The processor can run at up to **180 MHz**, enabling fast execution and high processing power for complex applications.
- **Harvard Architecture:**
Instead of using **one shared data bus**, it uses **multiple buses - Instruction**

(I-bus), Data (D-bus), and System (S-bus) - allowing simultaneous access to instructions and data for better performance.

3. Memory

- **Flash Memory:** 512 KB of on-chip Flash memory for program storage.
- **SRAM:** 128 KB of SRAM for data storage.

4. Interrupt System

- **Nested Vectored Interrupt Controller (NVIC):** Supports efficient handling of interrupts with 16 levels of priority.

5. Peripherals

- **Analog Peripherals:** Includes ADCs (Analog-to-Digital Converters), DACs (Digital-to-Analog Converters), and comparators.

6. Communication Interfaces:

- **UART/USART:** For asynchronous and synchronous serial communication.
- **SPI:** For synchronous serial communication with peripherals like sensors and memory devices.
- **I2C:** For synchronous communication with low-speed peripherals.

7. Debug and Trace

- **Embedded Trace Macrocell (ETM):**
Used to **trace program execution and identify bugs** in real time during debugging.

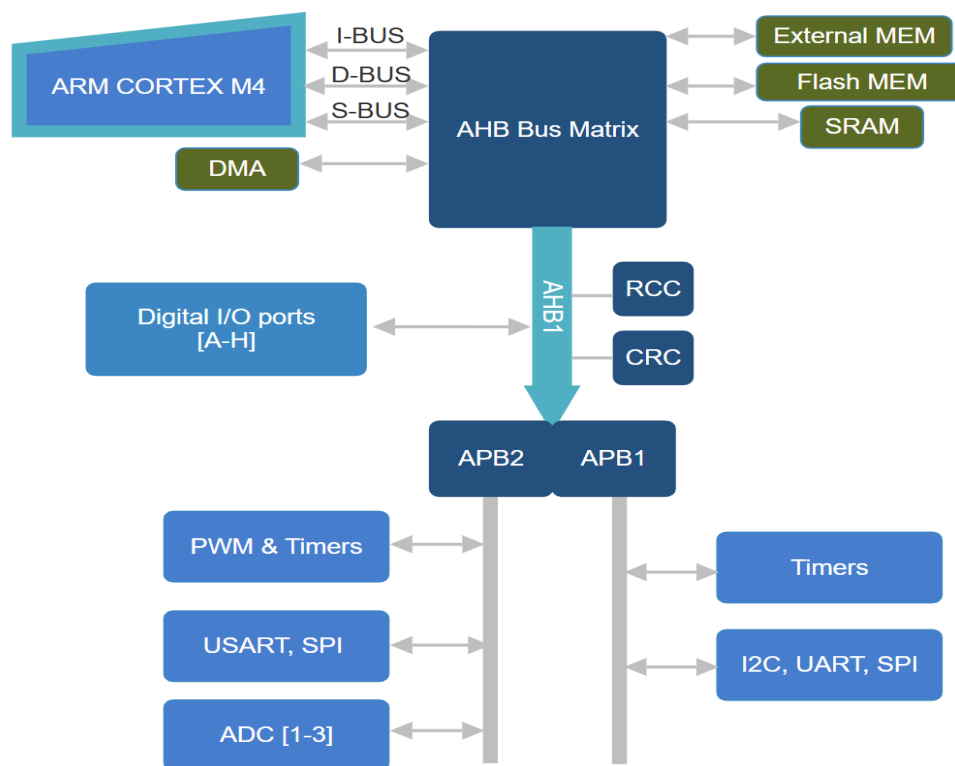
- **Serial Wire Debug (SWD):**

A **two-pin debugging interface** used for programming and debugging with fewer pins compared to JTAG.

Q. Which microcontroller feature is used for debugging and tracing?

-The **Debug and Trace unit** of the microcontroller, including **ETM (Embedded Trace Macrocell)** and **SWD (Serial Wire Debug)**, is used for program debugging and trace analysis.

System Architecture



- DMA: Direct Memory Access
- AHB: Advanced High-performance Bus

- APB: Advanced Peripheral Bus
- RCC: Reset and Clock Control
- CRC: Cyclic Redundancy Check
- PWM: Pulse with Modulation
- USART: universal synchronous/ asynchronous receiver/transmitter
- SPI: Serial Peripheral Interface
- I2C: Inter-Integrated Circuit
- ADC: Analog to Digital Converter

Memory and Bus Architecture

I-bus (Instruction Bus)

The I-bus connects the instruction bus of the Cortex-M4 with the FPU core to the Bus Matrix. This bus is used by the processor core to fetch instructions during program execution.

Key point: It controls the flow of instruction execution (most important).

D-bus (Data Bus)

The D-bus connects the data bus of the Cortex-M4 with the FPU core to the Bus Matrix. This bus is used by the processor core for literal load operations and debug access. The target of this bus is memory that contains code or data.

Key point: It is important for debugging.

S-bus (System Bus)

The S-bus connects the system bus of the Cortex-M4 with the FPU core to the Bus Matrix. This bus is used to access data stored in peripherals or SRAM. Instructions can also be fetched through this bus, but it is **less efficient** than the I-bus.

Key point: It handles all system access tasks.

Cyclic Redundancy Check (CRC)

CRC is a method used to detect errors in data. The CRC calculation unit takes a 32-bit data word and a fixed generator polynomial to create a CRC code. This code helps check if data was transmitted or stored correctly, **responsible for detecting error data and notifying the microcontroller about it.**

Reset and Clock Control (RCC)

RCC is a system that manages the enabling and disabling of ports and handles resetting their configurations.

There are three types of reset

1. System Reset
2. Power Reset
3. Backup domain Reset

***Reset and Clock Control (RCC) has **three main functions:**

1. **Enable** – Turns on the ports or peripherals.
2. **Configuration Reset** – Resets the settings of ports or peripherals to default.
3. **Power Reset** – Resets the power to specific parts of the system.

RCC AHB Peripheral Clock Enable Register [RCC→AHB1ENR]

- 32 bit register
- Can enable port any digital I/O port [A-H]
- Can enable DMAs
- Can enable CRC

Instruction 1:

$\text{RCC} \rightarrow \text{AHB1EN} | = 1 \ll 0$

[<< means left shift, **left-shift 1 by 0 bits**, so the output is 1]

Instruction 2:

$RCC \rightarrow AHB1EN \mid = 1 \ll 3$ [Output 1000]

Instruction 3:

$RCC \rightarrow AHB1EN \& = 1 \ll 5$ [Output 100000]

Note: If mentioned $\sim 1 \ll 5 = \sim 100000 = 011111$

7 6 5 4 3 2 1 0

H G F E D C B A

X X X X X X X X

1 [1ST INSTRUCTION]

X X X X X X X 1

1 0 0 0 [2ND INSTRUCTION]

X X X X 1 0 0 1

1 0 0 0 0 0 [3RD INSTRUCTION]

X X 1 0 0 0 0 0

Enable F port after executing these 3 instructions.

7 6 5 4 3 2 1 0
H G F E D C B A

X X X X X X X X

1 0 0 0 0 0 [3RD INSTRUCTION]

X X 1 0 0 0 0 0

1 0 0 0 [2ND INSTRUCTION]

X X 1 0 1 0 0 0

1 [1ST INSTRUCTION]

X X 1 0 1 0 0 1

Enable F, D, A Port.

GPIO Device Connection Configuration

#Connect an **input** device to **Port B, Pin 4**.

$RCC \rightarrow AHB1EN \mid = (1 \ll 1) \rightarrow \text{ACTIVE PORT B}$

$GPIOB \rightarrow MODER \mid = (00 \ll 8) \rightarrow \text{PIN 4 (INPUT} \rightarrow 00)$

Each pin uses **2 bits** in MODER, SO MULTIPLY BY 2 $\rightarrow 4 \times 2 = 8$

The MODER register in a GPIO port is used to select the operating mode of each pin.

#Connect an **output** device to **Port D, Pins 15, 13.**

RCC → AHB1EN |= (1 << 3) → ACTIVE PORT D

GPIOD → MODER |= (01 << 30) |= (01 << 26)

[2 BIT, SO MULTIPLY BY 2]

Pin 15 → $15 \times 2 = 30$

Pin 13 → $13 \times 2 = 26$

#Connect an **Input/Output** Device to **Port D, Pin 7**, but follows **SPI Protocol** (Direct Alternate Function)

If a **protocol** (SPI) is mentioned → use **Alternate Function mode (10).**

GPIOD → MODER |= (10 << 14) // Pin 7 → Alternate Function mode

Modes controlled by MODER

Bits	Mode
00	Input mode
01	General purpose output
10	Alternate function
11	Analog mode

OTYPER (Output Type Register)

Used to select output type (Push-Pull or Open-Drain).

#To **reduce the signal strength** for an output device on **Port D**, it should be set to **open-drain**.

$\text{GPIO} \rightarrow \text{OTYPER} \mid= (1 \ll 15) \mid= (1 \ll 13)$

[NO NEED TO MULTIPLY BY 2, because each pin uses **1 bit** only in OTYPER.]

Rule:

- If **reset state** is mentioned $\rightarrow$ use **OTYPER = 0** (Push-Pull).

Output types in OTYPER

Value	Output Type	Meaning
0	Push-Pull	Pin actively drives HIGH and LOW
1	Open-Drain	Pin can drive LOW or stay floating

OSPEEDR (Output Speed Register)

Speed = Frequency

#Set output speed of device on **Port D** to **Medium speed**:

$\text{GPIO} \rightarrow \text{OSPEEDR} \mid= (01 \ll 26) \mid= (01 \ll 30)$

(Each pin uses **2 bits**, so position = pin $\times$ 2)

Speed levels

Bits	Speed
00	Low speed
01	Medium speed
10	High speed
11	Very high speed

PUPDR (Pull-Up / Pull-Down Register)

Mainly used for input pins.

Bits	Configuration	Meaning
00	No pull-up, no pull-down	Pin is floating
01	Pull-up	Pin is connected internally to Vcc

10	Pull-down	Pin is connected internally to GND
11	Reserved	Not used

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

#Increase signal strength using **Pull-Up resistor** for input device on **Port B PIN 4**:

$\text{GPIOB} \rightarrow \text{PUPDR} |= (01 \ll 8)$

LCKR (Lock Register)

Used to lock configuration of a specific pin or entire port.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															LCKK
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

#Lock Pin 13

$\text{GPIOB} \rightarrow \text{LCKR} |= (1 \ll 13)$

#Lock Entire Port

If we set bit 1 in the 16-pin position, the entire port will be locked.

$\text{GPIOB} \rightarrow \text{LCKR} |= (1 \ll 16)$

Data Registers (Input Data Register)

IDR → **r**

GPIOx_IDR (x=A..H): These bits are read-only. They contain the input value of the corresponding I/O port.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 - Reserved

Bits 15:0 - IDRx: Port input data (y = 0..15)

ODR → **rw**

GPIOx_ODR (x=A..H): These bits can be read and written

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 - Reserved

Bits 15:0 - ODRy: Port output data (y = 0..15)

Set/Reset Register (GPIO BSRR)

The Set/Reset Register is used to set (make HIGH) or reset (make LOW) a GPIO pin without affecting other pins.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

- If we want to **set PIN 12** of any port, we write **BS12 = 1** (Bit Set 12).
- To **reset PIN 12**, the reset bit position is **12 + 16 = 28**.
- So, writing **BR12 = 1** (bit 28 = 1) will reset (clear) PIN 12.

Lower 16 bits (0–15): used for setting pins

Upper 16 bits (16–31): used for resetting pins

- **Set Pin 6 of Port B:**

`GPIOB→BSRR |= (1 << 6);` // Set pin 6 of Port B (make HIGH)

- **Reset Pin 6 of Port B:**

`GPIOB→BSRR |= (1 << 22);` // Reset pin 6 of Port B (6 + 16 = 22, make LOW)

Explanation:

- Bits **0–15** of BSRR → **Set pins**
- Bits **16–31** of BSRR → **Reset pins**
- Since reset bits start from 16, **pin number + 16** is used for resetting.

If output from PIN 12 is in reset state of Port C

`GPIOC→OTYPER |= (0 << 12);`

Simple Explanation:

- **OTYPER register** is used to select the **output type** of a GPIO pin.

- Writing **0** to bit 12 means **Push-Pull output type** (default state).
- So, **Pin 12 of Port C** is configured as **push-pull output**, not open-drain.

Note:

(0 << 12) does **not change/reset the pin output level**; it only defines the **output type**.

To reset (make LOW) the pin, the **BSRR reset bit** should be used.

Important Note: Reset State vs Reset (GPIO Registers)

If “reset state” is mentioned → use OTYPER register

- Reset state refers to the output configuration/type of the pin.
- The OTYPER register is used to select the output type (Push-Pull or Open-Drain).

Example:

```
GPIOC→OTYPER |= (0 << 12); // Pin 12 in reset state (Push-Pull output type)
```

Here, the pin configuration is defined, **not the output level**.

If “reset” is mentioned → use BSRR register

- Reset means making the pin output LOW (logic 0).
- The BSRR register is used to reset the pin safely.

Example:

```
GPIOC→BSRR |= (1 << 28); // Reset Pin 12 (12 + 16 = 28)
```

Here, the actual **pin output becomes LOW**.

So,

- **Reset state → OTYPER (configuration/type)**

- Reset → BSRR (output level LOW)

Alternate Function (AF) Configuration

- Each GPIO port has **16 pins**.
- Each pin needs **4 bits** to select an alternate function.
- So, total bits required = $4 \times 16 = 64$ bits.
- These 64 bits are divided into **2 registers**, each **32-bit** wide.

AFR Registers

- **AFR[0]** (AFRL - Alternate Function Low Register):
Used to configure Pin 0 to Pin 7.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

- **AFR[1]** (AFRH - Alternate Function High Register):
Used to configure Pin 8 to Pin 15.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRH15[3:0]				AFRH14[3:0]				AFRH13[3:0]				AFRH12[3:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRH11[3:0]				AFRH10[3:0]				AFRH9[3:0]				AFRH8[3:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Each pin uses **4 consecutive bits** inside these registers to select its alternate function.

Alternate Function Configuration Examples

ALT number 7 for Pin 4

`GPIOC→AFR[0] |= (7 << 16); // (4 × 4 = 16)`

Explanation:

- Pin 4 belongs to AFR[0] (pins 0–7).
- Each pin uses 4 bits.
- Bit position = Pin number $\times$ 4 = $4 \times 4 = 16$.

ALT number 7 for Pin 9

`GPIOC→AFR[1] |= (7 << 4);    // (9  $\times$  4 – 32 = 4)`

Explanation:

- Pin 9 belongs to AFR[1] (pins 8–15).
- AFR[1] starts after 32 bits, so shift position = (Pin $\times$ 4 – 32)
- Calculation: $9 \times 4 = 36 \rightarrow 36 - 32 = 4$.

Rule to Remember

- Pins 0–7 $\rightarrow$ AFR[0] $\rightarrow$ shift = pin $\times$ 4
- Pins 8–15 $\rightarrow$ AFR[1] $\rightarrow$ shift = (pin $\times$ 4 – 32)

Q. Suppose you are designing a smart home system using an **STM32F44GRE** microcontroller. You are trying to connect a **LDR sensor** using the **pin 4 of GPIO port F**. The data will be received from the LDR sensor with a **strong signal**. Moreover, you also have a temperature sensor that communicates via **I2C** and is connected to **pin 7 of port D for SCL** and **pin 8 of port D for SDA**. You have also connected an **LED light using pin 1 of port F**. After the temperature crosses a certain threshold, the microcontroller will turn on the LED light. The output data will be in **reset state** while being received by the LED.

The I2C Alternate Function number =3.

Now write down all the necessary **commands** to configure the system.

Configuration Commands for Smart Home System (STM32F446RE)

1. Enable GPIO Ports Clock

Enable Port F

`RCC→AHB1EN |= (1 << 5); // Enable clock for GPIOF`

Enable Port D

`RCC→AHB1EN |= (1 << 3); // Enable clock for GPIOD`

2. LDR Sensor (Input) - Port F, Pin 4

Set PF4 as Input Mode

`GPIOF→MODER |= (00 << 8); // (4 × 2 = 8)`

Strong Signal → Increase Signal Stability (Pull-up)

`GPIOF→PUPDR |= (01 << 8); // Pull-up enabled for PF4`

3. Temperature Sensor (I2C Protocol)

If a **protocol** is mentioned → pins must be in **Alternate Function mode**

I2C Pins:

- PD7 → SCL
- PD8 → SDA
- Alternate Function number = 3

Set PD7 and PD8 to Alternate Function Mode

$\text{GPIOD} \rightarrow \text{MODER} |= (10 \ll 14) |= (10 \ll 16); \quad // (7 \times 2 = 14), (8 \times 2 = 16)$

Configure Alternate Function = 3

For PD7 (AFR[0])

$\text{GPIOD} \rightarrow \text{AFR}[0] |= (3 \ll 28); \quad // (7 \times 4 = 28)$

For PD8 (AFR[1])

$\text{GPIOD} \rightarrow \text{AFR}[1] |= (3 \ll 0); \quad // (8 \times 4 - 32 = 0)$

4. LED Light (Output) - Port F, Pin 1

$\text{GPIOF} \rightarrow \text{MODER} |= (01 \ll 2); \quad // (1 \times 2 = 2)$

Output Data in Reset State

If **reset state** is mentioned $\rightarrow$ use **OTYPER**

$\text{GPIOF} \rightarrow \text{OTYPER} |= (0 \ll 1); \quad // \text{Push-pull (reset state configuration)}$